

torch.nn.functional.nll_loss#

https://pytorch.org/docs/stable/generated/torch.nn.functional.nll_loss.html

Description:

Compute the negative log likelihood loss. See NLLLoss for details. input (Tensor) – (N,C)(N, C)(N,C) where C = number of classes or (N,C,H,W)(N, C, H, W)(N,C,H,W) in case of 2D Loss, or (N,C,d1,d2,...,dK)(N, C, d_1, d_2, ..., d_K)(N,C,d1,d2,...,dK) where $K \geq 1$ in the case of K-dimensional loss. input is expected to be log-probabilities. target (Tensor) – (N)(N)(N) where each value is $0 \leq \text{targets}[i] \leq C-1$, or (N,d1,d2,...,dK)(N, d_1, d_2, ..., d_K)(N,d1,d2,...,dK) where $K \geq 1$ for K-dimensional loss. weight (Tensor, optional) – A manual rescaling weight given to each class. If given, has to be a Tensor of size C

Function Signature

```
torch.nn.functional.nll_loss(input, target, weight=None,
size_average=None, ignore_index=-100, reduce=None,
reduction='mean')[source]#
```

Parameters

Return type

Returns:

Tensor

Code Examples

```
>>> # input is of size N x C = 3 x 5
>>> input = torch.randn(3, 5, requires_grad=True)
>>> # each element in target has to have 0 <= value < C
>>> target = torch.tensor([1, 0, 4])
>>> output = F.nll_loss(F.log_softmax(input, dim=1), target)
>>> output.backward()
```

Detailed Documentation

Documentation

Compute the negative log likelihood loss.

See `NLLLoss` for details.

`input` (Tensor) – (N,C) (N, C) (N,C) where C = number of classes or (N,C,H,W) (N, C, H, W) (N,C,H,W) in case of 2D Loss, or $(N,C,d_1,d_2,\dots,d_K)$ $(N, C, d_1, d_2, \dots, d_K)$ $(N,C,d_1,d_2,\dots,d_K)$ where $K \geq 1$ $K \geq 1$ in the case of K -dimensional loss. `input` is expected to be log-probabilities.

`target` (Tensor) – (N) (N) (N) where each value is $0 \leq \text{targets}[i] \leq C-1$ $\leq \text{targets}[i] \leq C-1$, or $(N,d_1,d_2,\dots,d_K)$ $(N, d_1, d_2, \dots, d_K)$ $(N,d_1,d_2,\dots,d_K)$ where $K \geq 1$ $K \geq 1$ for K -dimensional loss.

`weight` (Tensor, optional) – A manual rescaling weight given to each class. If given, has to be a Tensor of size C

`size_average` (bool, optional) – Deprecated (see reduction).

`ignore_index` (int, optional) – Specifies a target value that is ignored and does not contribute to the input gradient. When `size_average` is `True`, the loss is averaged over non-ignored targets. Default: -100

`reduce` (bool, optional) – Deprecated (see reduction).

`reduction` (str, optional) – Specifies the reduction to apply to the output: 'none' | 'mean' | 'sum'. 'none': no reduction will be applied, 'mean': the sum of the output will be divided by the number of elements in the output, 'sum': the output will be summed. Note: `size_average` and `reduce` are in the process of being deprecated, and in the meantime, specifying either of those two args will override `reduction`. Default: 'mean'

